National University of Sciences and Technology
(NUST)
School of Electrical Engineering and Computer Science

Lab 09
Type Checking and Scope Checking

Name	Abrar Butt
Qalam ID	460297
Course	CS-346 Compiler Construction
Class	BSCS-2023-CD
Date	9 April 2026
Lab Instructor	Ms. Urooj Akmal
Instructor	Miss Yusra

Faculty of Computing

The complete source code for this lab is available on GitHub:

https://github.com/Abrarbut/compiler-construction

Contents

1 Introduction **2**

2 Objectives **2**

3 Tools and Software **2**

4 Background **2**
4.1 Type Checking 2
4.2 Type Casting 2
4.3 Scope Checking 2

5 Task 1 – Type Checking and Type Casting **3**
5.1 Description 3
5.2 Source Code 3
5.3 Program Output 5
5.4 Analysis 6

6 Task 2 – Scope Checking (Scanner + Parser) **6**
6.1 Description 6
6.2 Source Code 7
6.3 Program Output 13
6.4 Analysis 14

7 Conclusion **14**

Introduction

Parsing ensures that a program follows syntactic rules, confirming that tokens are arranged in a valid structure. However, syntactic correctness alone does not guarantee that a program is meaningful. This is where **semantic analysis** becomes essential.

Semantic analysis verifies that program statements are logically consistent within the rules of the programming language. Key responsibilities include:

- Variables, functions, and classes must be properly declared before use.
- Types must be used correctly in expressions.
- Scope and visibility rules must be respected.
- Access control must be enforced.

Semantic analysis is the **second-to-last phase of the compiler front-end** and acts as the final checkpoint before code generation.

Objectives

1. Implement a program that demonstrates **type checking** and **type casting**.
2. Implement a program for **scope checking** incorporating a lexical scanner and parser.

Tools and Software

- **GCC** (GNU Compiler Collection) on Linux/Windows – C99 standard.

Background

Type Checking

Type checking ensures that operations are performed on compatible data types. Assigning a `float` to an `int` without an explicit cast loses precision and may be flagged as a compilation error.

Type Casting

Type casting is the conversion of a value from one data type to another. **Implicit (widening)** promotion (e.g. `char` → `int`) is automatic, while **explicit (narrowing)** casts (e.g. `float` → `int`) must be written by the programmer and truncate the fractional part.

Scope Checking

Scope checking determines the visibility of identifiers within different regions of a program. A *scoped symbol table* is maintained as a stack: entering a block pushes a new scope level; exiting pops it. During lookup, the innermost scope is searched first, so a local variable *shadows* a global variable of the same name.

Task 1 – Type Checking and Type Casting

Description

The program maintains a simple symbol table with three variables of types `int`, `float`, and `char`. It performs type checking for several assignment scenarios, then demonstrates four explicit type casts.

Source Code

```c
/*
 * Lab 09 - Task 1: Type Checking and Type Casting
 * Course: CS-346 Compiler Construction  |  NUST - SEECS
 * Compile: gcc task1_type_checker.c -o task1  &&  ./task1
 */
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

/* Data-type enumeration & Symbol structure */
typedef enum { TYPE_INT, TYPE_FLOAT, TYPE_CHAR, TYPE_UNKNOWN } DataType;

typedef struct {
    char name[50];
    DataType type;
    union {
        int   ival;
        float fval;
        char  cval;
    } value;
} Symbol;

const char* typeName(DataType t) {
    switch (t) {
        case TYPE_INT:   return "int";
        case TYPE_FLOAT: return "float";
        case TYPE_CHAR:  return "char";
        default:         return "unknown";
    }
}

/*
 * typeCheck -- returns 1 if assigning 'rhs' type to 'lhs' type
 * is compatible without an explicit cast, 0 otherwise.
 */
int typeCheck(DataType lhs, DataType rhs) {
    if (lhs == rhs)                              return 1; /* same type    */
    if (lhs == TYPE_INT && rhs == TYPE_CHAR)     return 1; /* char -> int  */
    return 0;                                              /* needs cast   */
}

/*
 * typeCast -- explicitly converts src to targetType,
```

```c
 * prints what happened, and returns the new Symbol.
 */
Symbol typeCast(Symbol src, DataType targetType) {
    Symbol result;
    strcpy(result.name, src.name);
    result.type = targetType;

    if (src.type == TYPE_FLOAT && targetType == TYPE_INT) {
        result.value.ival = (int)src.value.fval;
        printf("  [Cast] float %.4f  -->  int %d  "
               "(fractional part truncated)\n",
               src.value.fval, result.value.ival);
    } else if (src.type == TYPE_INT && targetType == TYPE_FLOAT) {
        result.value.fval = (float)src.value.ival;
        printf("  [Cast] int %d  -->  float %.4f\n",
               src.value.ival, result.value.fval);
    } else if (src.type == TYPE_CHAR && targetType == TYPE_INT) {
        result.value.ival = (int)src.value.cval;
        printf("  [Cast] char '%c' (ASCII %d)  -->  int %d\n",
               src.value.cval, src.value.cval, result.value.ival);
    } else if (src.type == TYPE_INT && targetType == TYPE_CHAR) {
        result.value.cval = (char)src.value.ival;
        printf("  [Cast] int %d  -->  char '%c'\n",
               src.value.ival, result.value.cval);
    } else {
        printf("  [Cast Error] No valid cast from %s to %s\n",
               typeName(src.type), typeName(targetType));
        result = src;
    }
    return result;
}

void reportTypeCheck(const char* lName, DataType lType,
                     const char* rName, DataType rType) {
    printf("  Assigning %-5s (%s) = %-5s (%s)  -->  ",
           lName, typeName(lType), rName, typeName(rType));
    if (typeCheck(lType, rType))
        printf("PASSED\n");
    else
        printf("TYPE ERROR: incompatible (explicit cast required)\n"
            );
}

int main(void) {
    printf("+=========================================+\n");
    printf("|  Lab 09 - Task 1: Type Checking & Casting|\n");
    printf("+=========================================+\n\n");

    Symbol varInt;   strcpy(varInt.name,   "a");
    varInt.type = TYPE_INT;  varInt.value.ival = 42;

    Symbol varFloat; strcpy(varFloat.name, "b");
    varFloat.type = TYPE_FLOAT; varFloat.value.fval = 9.81f;

    Symbol varChar;  strcpy(varChar.name,  "c");
    varChar.type = TYPE_CHAR;  varChar.value.cval = 'Z';

    printf("--- Symbol Table ---------------------------\n");
```

```c
    printf("  %-6s  %-6s  %s\n", "Name", "Type", "Value");
    printf("  %-6s  %-6s  %d\n",
           varInt.name,   typeName(varInt.type),   varInt.value.ival);
    printf("  %-6s  %-6s  %.4f\n",
           varFloat.name, typeName(varFloat.type), varFloat.value.fval);
    printf("  %-6s  %-6s  '%c'\n",
           varChar.name,  typeName(varChar.type),  varChar.value.cval);
    printf("\n");

    printf("--- Type Checking --------------------------\n");
    reportTypeCheck("a", TYPE_INT,   "a", TYPE_INT);   /* int   = int   OK */
    reportTypeCheck("a", TYPE_INT,   "c", TYPE_CHAR);  /* int   = char   OK */
    reportTypeCheck("a", TYPE_INT,   "b", TYPE_FLOAT); /* int   = float ERROR */
    reportTypeCheck("b", TYPE_FLOAT, "a", TYPE_INT);   /* float= int   ERROR */
    printf("\n");

    printf("--- Type Casting ---------------------------\n");

    printf("(1) float -> int:\n");
    Symbol r1 = typeCast(varFloat, TYPE_INT);
    printf("    Result: %s %s = %d\n\n",
           typeName(r1.type), r1.name, r1.value.ival);

    printf("(2) int -> float:\n");
    Symbol r2 = typeCast(varInt, TYPE_FLOAT);
    printf("    Result: %s %s = %.4f\n\n",
           typeName(r2.type), r2.name, r2.value.fval);

    printf("(3) char -> int:\n");
    Symbol r3 = typeCast(varChar, TYPE_INT);
    printf("    Result: %s %s = %d\n\n",
           typeName(r3.type), r3.name, r3.value.ival);

    printf("(4) int -> char:\n");
    Symbol r4 = typeCast(varInt, TYPE_CHAR);
    printf("    Result: %s %s = '%c'\n\n",
           typeName(r4.type), r4.name, r4.value.cval);

    printf("--- End of Task 1 --------------------------\n");
    return 0;
}
```

Listing 1: task1_type_checker.c

Program Output

Listing 2: Output – task1_type_checker

```
+=========================================+
|  Lab 09 - Task 1: Type Checking & Casting |
+=========================================+
```

```
--- Symbol Table ---------------------------
  Name     Type     Value
  a        int      42
  b        float    9.8100
  c        char     'Z'

--- Type Checking --------------------------
  Assigning a     (int) = a     (int)  -->  PASSED
  Assigning a     (int) = c     (char)  -->  PASSED
  Assigning a     (int) = b     (float)  -->  TYPE ERROR: incompatible
     types (explicit cast required)
  Assigning b     (float) = a     (int)  -->  TYPE ERROR: incompatible
     types (explicit cast required)

--- Type Casting ---------------------------
(1) float -> int:
  [Cast] float 9.8100  -->  int 9  (fractional part truncated)
      Result: int b = 9

(2) int -> float:
  [Cast] int 42  -->  float 42.0000
      Result: float a = 42.0000

(3) char -> int:
  [Cast] char 'Z' (ASCII 90)  -->  int 90
      Result: int c = 90

(4) int -> char:
  [Cast] int 42  -->  char '*'
      Result: char a = '*'

--- End of Task 1 --------------------------
```

Analysis

- **int** = **int** and **int** = **char**: pass directly – same type or safe widening.
- **int** = **float** and **float** = **int**: rejected – narrowing requires an explicit cast.
- `float 9.81` cast to `int` yields `9` (truncation, not rounding).
- `char 'Z'` (ASCII 90) cast to `int` yields `90`.
- `int 42` cast to `char` yields `'*'` (ASCII 42).

Task 2 – Scope Checking (Scanner + Parser)

Description

A three-stage front-end pipeline is implemented in C:

1. **Scanner (Lexer)** – tokenises source code into a typed token stream, recording line numbers and discarding comments.
2. **Recursive-descent Parser** – recognises variable declarations, function definitions, `cout` statements, and `return` statements.

3. **Scoped Symbol Table** – a stack of scope levels. `scopeDeclare()` adds to the current level; `scopeLookup()` searches innermost-first so that local variables shadow globals.

The input analysed is the exact C++-style snippet provided in the lab:

```
// Global variable declaration
int g = 20;

int main() {
    // Local variable declaration
    int g = 10;
    cout << g;
    return 0;
}
```

Listing 3: Input source code under analysis

Source Code

```
/*
 * Lab 09 - Task 2: Scope Checking (Scanner + Parser)
 * Course: CS-346 Compiler Construction  |  NUST - SEECS
 * Compile: gcc task2_scope_checker.c -o task2  &&  ./task2
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

/* ================================================
   SECTION 1 -- LEXER / SCANNER
   ================================================ */
typedef enum {
    TOK_INT, TOK_FLOAT, TOK_RETURN,
    TOK_IDENT, TOK_NUMBER,
    TOK_ASSIGN, TOK_SEMICOLON,
    TOK_LBRACE, TOK_RBRACE,
    TOK_LPAREN, TOK_RPAREN,
    TOK_LSHIFT, TOK_COMMA,
    TOK_COUT, TOK_EOF, TOK_UNKNOWN
} TokenType;

typedef struct {
    TokenType type;
    char      lexeme[64];
    int       line;
} Token;

static const char* src;
static int srcPos, lineNo;

void lexerInit(const char* code) {
    src = code; srcPos = 0; lineNo = 1;
}

static void skipWS(void) {
    for (;;) {
```

```
        while (src[srcPos] && isspace(src[srcPos])) {
            if (src[srcPos] == '\n') lineNo++;
            srcPos++;
        }
        if (src[srcPos]=='/' && src[srcPos+1]=='/') {
            while (src[srcPos] && src[srcPos]!='\n') srcPos++;
            continue;
        }
        break;
    }
}

Token lexerNext(void) {
    Token tok; tok.lexeme[0]='\0'; tok.line=lineNo;
    skipWS();
    if (!src[srcPos]) { tok.type=TOK_EOF;
                        strcpy(tok.lexeme,"EOF"); return tok; }

    if (isalpha(src[srcPos]) || src[srcPos]=='_') {
        int i=0;
        while (isalnum(src[srcPos])||src[srcPos]=='_')
            tok.lexeme[i++]=src[srcPos++];
        tok.lexeme[i]='\0';
        if      (!strcmp(tok.lexeme,"int"))     tok.type=TOK_INT;
        else if (!strcmp(tok.lexeme,"float"))   tok.type=TOK_FLOAT;
        else if (!strcmp(tok.lexeme,"return")) tok.type=TOK_RETURN;
        else if (!strcmp(tok.lexeme,"cout"))    tok.type=TOK_COUT;
        else                                    tok.type=TOK_IDENT;
        return tok;
    }
    if (isdigit(src[srcPos])) {
        int i=0;
        while (isdigit(src[srcPos])||src[srcPos]=='.')
            tok.lexeme[i++]=src[srcPos++];
        tok.lexeme[i]='\0'; tok.type=TOK_NUMBER; return tok;
    }
    if (src[srcPos]=='<' && src[srcPos+1]=='<') {
        strcpy(tok.lexeme,"<<");
        tok.type=TOK_LSHIFT; srcPos+=2; return tok;
    }
    tok.lexeme[0]=src[srcPos]; tok.lexeme[1]='\0';
    switch (src[srcPos++]) {
        case '=': tok.type=TOK_ASSIGN;    break;
        case ';': tok.type=TOK_SEMICOLON; break;
        case '{': tok.type=TOK_LBRACE;    break;
        case '}': tok.type=TOK_RBRACE;    break;
        case '(': tok.type=TOK_LPAREN;    break;
        case ')': tok.type=TOK_RPAREN;    break;
        default:  tok.type=TOK_UNKNOWN;   break;
    }
    return tok;
}

#define MAX_TOKENS 512
static Token toks[MAX_TOKENS];
static int tokCount=0, tokCur=0;

void tokenize(const char* code) {
```

```c
    lexerInit(code); Token t;
    do { t=lexerNext(); toks[tokCount++]=t; }
    while (t.type!=TOK_EOF && tokCount<MAX_TOKENS-1);
}

void printTokens(void) {
    printf("--- Token Stream ---------------------------\n");
    printf("  %-4s  %-12s  %s\n","Line","Type","Lexeme");
    for (int i=0;i<tokCount;i++) {
        const char* tn="UNKNOWN";
        switch(toks[i].type){
            case TOK_INT:       tn="INT";       break;
            case TOK_FLOAT:     tn="FLOAT";     break;
            case TOK_RETURN:    tn="RETURN";    break;
            case TOK_IDENT:     tn="IDENT";     break;
            case TOK_NUMBER:    tn="NUMBER";    break;
            case TOK_ASSIGN:    tn="ASSIGN";    break;
            case TOK_SEMICOLON: tn="SEMICOLON"; break;
            case TOK_LBRACE:    tn="LBRACE";    break;
            case TOK_RBRACE:    tn="RBRACE";    break;
            case TOK_LPAREN:    tn="LPAREN";    break;
            case TOK_RPAREN:    tn="RPAREN";    break;
            case TOK_LSHIFT:    tn="LSHIFT";    break;
            case TOK_COUT:      tn="COUT";      break;
            case TOK_EOF:       tn="EOF";       break;
            default: break;
        }
        printf("  %-4d  %-12s  \"%s\"\n",
               toks[i].line, tn, toks[i].lexeme);
    }
    printf("\n");
}

static Token peekTok(void)     { return toks[tokCur]; }
static Token consumeTok(void)  { return toks[tokCur++]; }
static Token expectTok(TokenType t) {
    Token tk=consumeTok();
    if (tk.type!=t)
        printf("  [Parse Error] Line %d: unexpected \"%s\"\n",
               tk.line, tk.lexeme);
    return tk;
}
```

Listing 4: task2_scope_checker.c – Section 1: Lexer

```c
/* ================================================
   SECTION 2 -- SCOPED SYMBOL TABLE
   ================================================ */
#define MAX_SCOPE_DEPTH    16
#define MAX_SYMS_PER_SCOPE 64

typedef struct { char name[64]; char type[16]; char value[64]; }
    SymEntry;
typedef struct {
    SymEntry entries[MAX_SYMS_PER_SCOPE];
    int count;
    char label[32];
} ScopeLevel;
```

```c

static ScopeLevel scopeStack[MAX_SCOPE_DEPTH];
static int scopeDepth = -1;

void scopeEnter(const char* label) {
    scopeDepth++;
    scopeStack[scopeDepth].count = 0;
    strncpy(scopeStack[scopeDepth].label, label, 31);
    printf("  [Scope] >>> Enter scope: \"%s\"\n", label);
}
void scopeExit(void) {
    printf("  [Scope] <<< Exit  scope: \"%s\"\n",
           scopeStack[scopeDepth].label);
    scopeDepth--;
}

/* Declare in the CURRENT (innermost) scope */
int scopeDeclare(const char* name, const char* type,
                 const char* value) {
    ScopeLevel* cur = &scopeStack[scopeDepth];
    for (int i = 0; i < cur->count; i++) {
        if (!strcmp(cur->entries[i].name, name)) {
            printf("  [Error] Redeclaration of \"%s\" in \"%s\"\n",
                   name, cur->label);
            return 0;
        }
    }
    SymEntry* e = &cur->entries[cur->count++];
    strncpy(e->name, name, 63);
    strncpy(e->type, type, 15);
    strncpy(e->value, value, 63);
    printf("  [Declare] %-6s %-6s = %-6s  (scope: \"%s\")\n",
           type, name, value, cur->label);
    return 1;
}

/* Lookup -- innermost scope wins */
SymEntry* scopeLookup(const char* name) {
    for (int d = scopeDepth; d >= 0; d--) {
        ScopeLevel* sc = &scopeStack[d];
        for (int i = 0; i < sc->count; i++)
            if (!strcmp(sc->entries[i].name, name))
                return &sc->entries[i];
    }
    return NULL;
}
```

Listing 5: task2_scope_checker.c – Section 2: Symbol Table

```c
/* ================================================
   SECTION 3 -- RECURSIVE-DESCENT PARSER
   ================================================ */
static void parseBlock(void);

static void parseVarDecl(void) {
    Token typeTok = consumeTok();
    Token nameTok = expectTok(TOK_IDENT);
    expectTok(TOK_ASSIGN);
```

```
    Token valTok  = consumeTok();
    expectTok(TOK_SEMICOLON);
    scopeDeclare(nameTok.lexeme, typeTok.lexeme, valTok.lexeme);
}

static void parseCout(void) {
    consumeTok();                /* cout */
    expectTok(TOK_LSHIFT);       /* <<   */
    Token exprTok = consumeTok();
    if (exprTok.type == TOK_IDENT) {
        SymEntry* e = scopeLookup(exprTok.lexeme);
        if (e) {
            printf("  [cout] \"%s\" resolved to value = %s\n",
                   exprTok.lexeme, e->value);
            for (int d = scopeDepth; d >= 0; d--) {
                ScopeLevel* sc = &scopeStack[d];
                for (int i = 0; i < sc->count; i++) {
                    if (!strcmp(sc->entries[i].name,
                                exprTok.lexeme)) {
                        printf("         (declared in scope"
                               " \"%s\")\n", sc->label);
                        goto found;
                    }
                }
            }
            found:;
        } else {
            printf("  [Error] Undeclared variable \"%s\"\n",
                   exprTok.lexeme);
        }
    }
    while (peekTok().type!=TOK_SEMICOLON &&
           peekTok().type!=TOK_EOF) consumeTok();
    expectTok(TOK_SEMICOLON);
}

static void parseReturn(void) {
    consumeTok();
    while (peekTok().type!=TOK_SEMICOLON &&
           peekTok().type!=TOK_EOF) consumeTok();
    expectTok(TOK_SEMICOLON);
}

static void parseBlock(void) {
    expectTok(TOK_LBRACE);
    while (peekTok().type!=TOK_RBRACE &&
           peekTok().type!=TOK_EOF) {
        Token t = peekTok();
        if      (t.type==TOK_INT || t.type==TOK_FLOAT) parseVarDecl
            ();
        else if (t.type==TOK_COUT)   parseCout();
        else if (t.type==TOK_RETURN) parseReturn();
        else consumeTok();
    }
    expectTok(TOK_RBRACE);
}

static void parseFunction(void) {
```

```
    consumeTok();
    Token nameTok = consumeTok();
    expectTok(TOK_LPAREN); expectTok(TOK_RPAREN);
    scopeEnter(nameTok.lexeme);
    parseBlock();
    scopeExit();
}

void parse(void) {
    scopeEnter("global");
    while (peekTok().type != TOK_EOF) {
        Token t = peekTok();
        if (t.type==TOK_INT || t.type==TOK_FLOAT) {
            if (tokCur+2 < tokCount &&
                toks[tokCur+2].type == TOK_LPAREN)
                parseFunction();
            else
                parseVarDecl();
        } else { consumeTok(); }
    }
    scopeExit();
}

/* ================================================
   SECTION 4 -- MAIN
   ================================================ */
int main(void) {
    printf("+==========================================+\n");
    printf("|  Lab 09 - Task 2: Scope Checker          |\n");
    printf("|  (Scanner + Parser + Symbol Table)       |\n");
    printf("+==========================================+\n\n");

    const char* code =
        "// Global variable declaration\n"
        "int g = 20;\n"
        "int main() {\n"
        "    // Local variable declaration\n"
        "    int g = 10;\n"
        "    cout << g;\n"
        "    return 0;\n"
        "}\n";

    printf("--- Input Source Code ----------------------\n");
    printf("%s\n", code);

    printf("--- Step 1: Scanning -----------------------\n");
    tokenize(code);
    printTokens();

    printf("--- Step 2: Parsing & Scope Analysis -------\n");
    parse();

    printf("\n--- Result ---------------------------------\n");
    printf("  Global : int g = 20\n");
    printf("  Local  : int g = 10  (inside main)\n");
    printf("  cout << g  -->  10\n");
    printf("  (local variable shadows the global)\n");
    printf("--- End of Task 2 --------------------------\n");
```

```
    return 0;
}
```

Listing 6: task2_scope_checker.c – Section 3 & 4: Parser + Main

Program Output

Listing 7: Output – task2_scope_checker

```
+==========================================+
|  Lab 09 - Task 2: Scope Checker          |
|  (Scanner + Parser + Scoped Symbol Table) |
+==========================================+

--- Input Source Code ----------------------
// Global variable declaration
int g = 20;

int main() {
    // Local variable declaration
    int g = 10;
    cout << g;
    return 0;
}

--- Step 1: Scanning (Tokenisation) --------
--- Token Stream ---------------------------
  Line  Type          Lexeme
  1     INT           "int"
  2     IDENT         "g"
  2     ASSIGN        "="
  2     NUMBER        "20"
  2     SEMICOLON     ";"
  2     INT           "int"
  4     IDENT         "main"
  4     LPAREN        "("
  4     RPAREN        ")"
  4     LBRACE        "{"
  4     INT           "int"
  6     IDENT         "g"
  6     ASSIGN        "="
  6     NUMBER        "10"
  6     SEMICOLON     ";"
  6     COUT          "cout"
  7     LSHIFT        "<<"
  7     IDENT         "g"
  7     SEMICOLON     ";"
  7     RETURN        "return"
  8     NUMBER        "0"
  8     SEMICOLON     ";"
  8     RBRACE        "}"
  9     EOF           "EOF"

--- Step 2: Parsing & Scope Analysis -------
  [Scope] >>> Enter scope: "global"
  [Declare] int    g      = 20      (scope: "global")
  [Scope] >>> Enter scope: "main"
  [Declare] int    g      = 10      (scope: "main")
```

```
  [cout] "g" resolved to value = 10  (from scope "main")
         (declared in scope "main")
  [Scope] <<< Exit  scope: "main"
  [Scope] <<< Exit  scope: "global"

--- Scope Check Result ---------------------
  Global  : int g = 20
  Local   : int g = 10  (inside main)
  Output  : cout << g  -->  10
  Reason  : Local variable shadows global variable.
            Innermost scope takes precedence.
--- End of Task 2 --------------------------
```

Analysis

- **Scanner**: every construct is correctly tokenised, including keywords, identifiers, numeric literals, the « operator, and braces. Single-line comments are silently discarded.
- **Global scope**: `int g = 20` is registered in the `"global"` scope.
- **Local scope**: entering `main()` pushes a new scope; `int g = 10` is registered in the `"main"` scope.
- **Lookup**: when `cout « g` is evaluated, the stack is searched from the innermost (`"main"`) outward. The local entry (`g = 10`) is found first; the global `g = 20` is never reached.
- This correctly implements C/C++ shadowing: a local variable with the same name overrides the global within its enclosing scope.

Conclusion

This lab demonstrated two core responsibilities of semantic analysis in compiler construction.

Task 1 showed that a type checker must verify compatibility before any assignment or operation. Incompatible combinations (e.g. `float` assigned to `int`) are flagged as type errors, and explicit casts are needed to resolve them. Casting may involve narrowing (loss of the fractional part) or widening (safe promotion).

Task 2 illustrated how a compiler resolves identifiers in the presence of nested scopes through a three-stage pipeline: *scan* → *parse* → *scope check*. Maintaining a scope stack ensures that local declarations correctly shadow global ones – the standard behaviour observed in C and C++.

Together, type checking and scope checking ensure that a program is not only syntactically correct but also logically consistent, allowing the compiler to proceed safely to intermediate code generation.